Boosting!

Agenda

- At a high level, what is boosting and what problem does it solve?
- Why do we mostly only boost trees?
- The AdaBoost model and algorithm
- The Gradient Boost model and algorithm
- Python!

 © 2022 General Assembly

Remind me...

Why (so far) have we used **ensemble** methods such as bagging and random forests? What problem did this solve?



 © 2022 General Assembly

Bagging and Random Forests

We ensembled overfit decision trees in order to reduce variance. That is, **averaging overfit models together was a way to reduce the overfitedness of the overall model.**

An added bonus was that these ensemble models are **parallelizable**. Why is that?



 © 2022 General Assembly

Boosting

In contrast, **boosting** is another ensemble technique that solves the opposite problem:

We're going to take **weak learners** and successively **boost** them to **reduce bias.**

Typically, these weak learners are going to be very high-bias decision trees. Specifically, they're most often 1-split **decision stumps**. (ie, `max_depth=1`)

 © 2022 General Assembly

random forest

overfit trees

bootstrapping and averaging

underfit “stumps”

AdaBoost

boosting

underfit
(high bias)

just right
(good tradeoff)

overfit
(high variance)

Ok, but why trees?

Graphic stolen from our Rosarium philosophorum, *The Elements of Statistical Learning*, by Hastie, Tibshirani, and Friedman (praise be unto them)

TABLE 10.1. *Some characteristics of different learning methods. Key:* ▲ *= good,* ◆ *=fair, and* ▼ *=poor.*

Characteristic	Neural Nets	SVM	Trees	MARS	k-NN, Kernels
Natural handling of data of "mixed" type	▼	▼	▲	▲	▼
Handling of missing values	▼	▼	▲	▲	▲
Robustness to outliers in input space	▼	▼	▲	▼	▲
Insensitive to monotone transformations of inputs	▼	▼	▲	▼	▼
Computational scalability (large N)	▼	▼	▲	▲	▼
Ability to deal with irrelevant inputs	▼	▼	▲	▲	▼
Ability to extract linear combinations of features	▲	▲	▼	▼	◆
Interpretability	▼	▼	◆	▲	▼
Predictive power	▲	▲	▼	◆	▲

 © 2022 General Assembly

AdaBoost

(math inbound)

AdaBoost

The “simplest” (lol) form of boosting is known as the **AdaBoost**.

The general idea of boosting is to fit successive models that each improve upon the one before them.

After each model step in the AdaBoost, the **next model pays more attention to the parts of the data it got the most incorrect**. By focusing special attention on these parts, it spends most of its time learning the hardest parts of your data rather than the easiest, most separable parts.

 © 2022 General Assembly

AdaBoost Algorithm (as math-free as possible)

1. Initialize the observation weights **w** to each be 1/n (ie, all weighted the same)
2. For $m = 1, \ldots, M$:
   a. Fit a ***weighted*** underfit model G_m
   b. Compute the ***weighted*** misclassification rate
   c. Compute the model weight α_m that determines this model's overall importance to the ensemble
   d. Reweigh the observations as weight vector **w**. This is a vector in which the "easy" observations have a low weight and the "hard" observations get a large weight
3. The final model G is the weighted amalgamation of all the G_ms and the α_ms.

 © 2022 General Assembly

AdaBoost (mathfull notation)

When talking about boosting (and also for SVMs later), when we talk about binary classification, we let $y \in \{-1, 1\}$ instead of the usual $y \in \{0, 1\}$ so we can take advantage of a few math tricks.

In what follows...

- $G_m(x)$ is a model, and so it can take only values -1 and 1
- $I(.)$ is the indicator function. ie, $I(A)$ is 1 if A is true and 0 otherwise.
- sign(.) is the “signum” function. ie, sign(x) = 1 if x is positive and -1 if it’s negative.

 © 2022 General Assembly

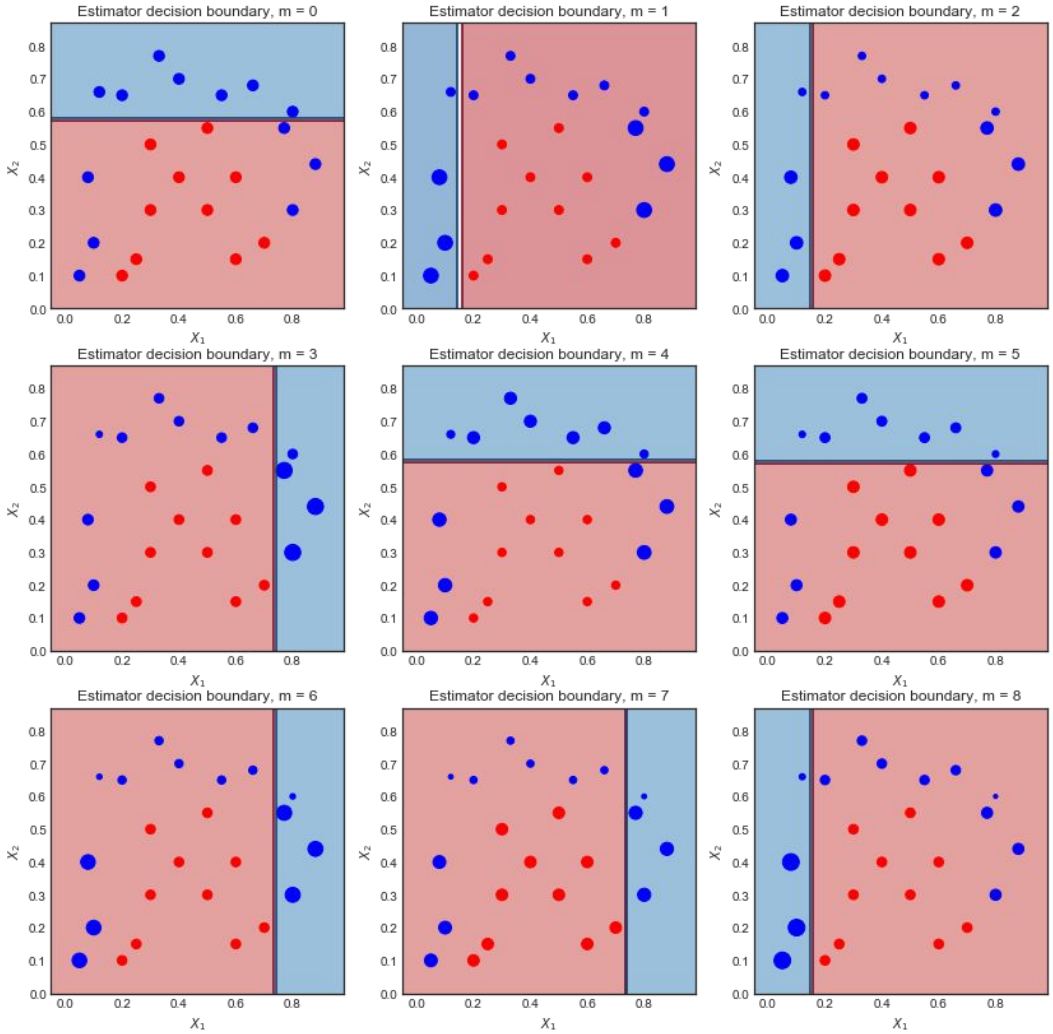
AdaBoost Algorithm (for all you math nerds out there)

Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N,\ i = 1, 2, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit a classifier $G_m(x)$ to the training data using weights w_i.
   (b) Compute
   $$\text{err}_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}.$$
   (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
   (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))],\ i = 1, 2, \ldots, N$.
3. Output $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

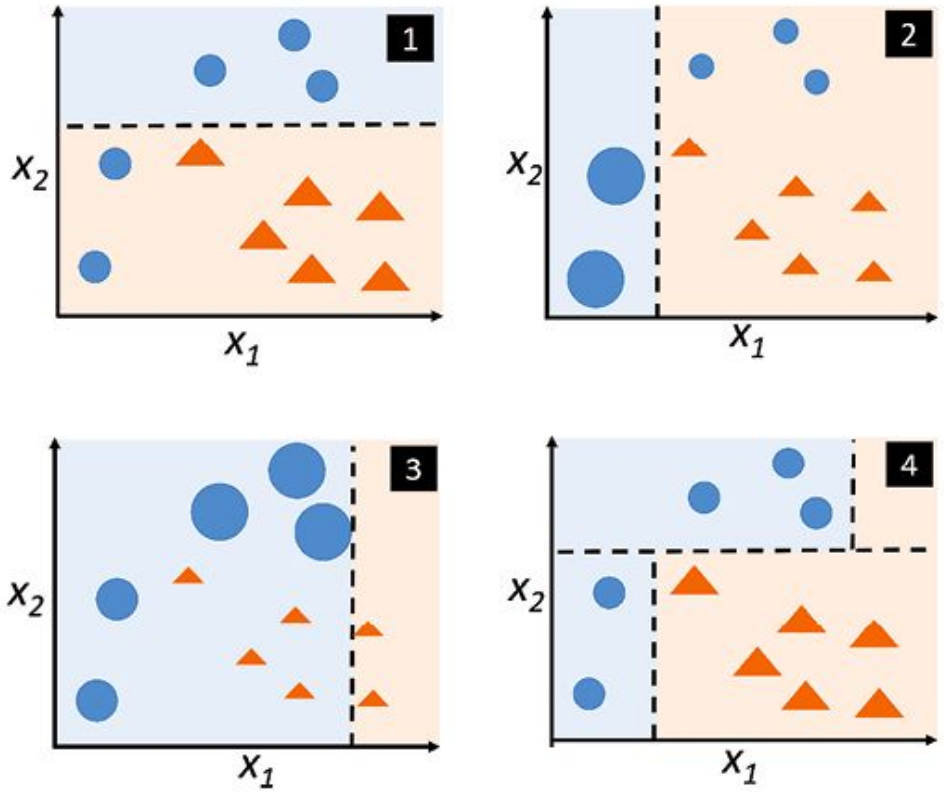
(As always, clipped directly from ESL)

AdaBoost Viz



 © 2022 General Assembly

Another Viz



Gradient Boost

(oh God there's more math and it's worse somehow)

AdaBoost vs Gradient Boost

Both are similar boosting techniques, but gradient boost takes a slightly different approach.

Where AdaBoost improves successive models by focusing more on the hard-to-reach parts of our data...

Gradient boost focuses on the observations with the worst residuals and successively corrects the model to "fix" them.

 © 2022 General Assembly

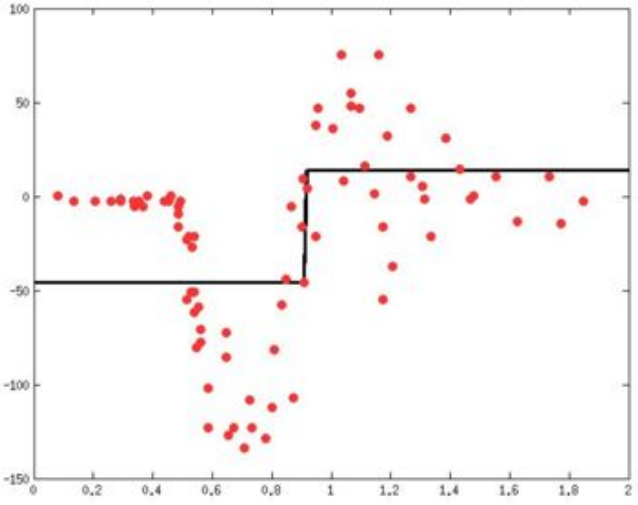
The Gradient Tree Boost Algorithm (math lovers keep OUT)

1. Initialize the base model G_0 to be the null model.
2. For $m = 1, \ldots, M$:
    a. Compute the ~~gradient of the loss function~~ "residuals", **r**.
        i. There's a special way of computing residuals for classification problems called **deviance**, and let us never speak of it again!
    b. Fit an underfit tree model, except with **r** instead of **y** as the response variable.
    c. For each terminal region (ie, partition of feature space) computed in 2b, compute a new residual γ that gets you the best loss function with respect to $f_{m\text{-}1} + \gamma$ (ie, this is readjusting the predictions based on how bad they were)
    d. Update $f_m = f_{m\text{-}1}$ + adjustments involving γ
3. The final model is just f_M

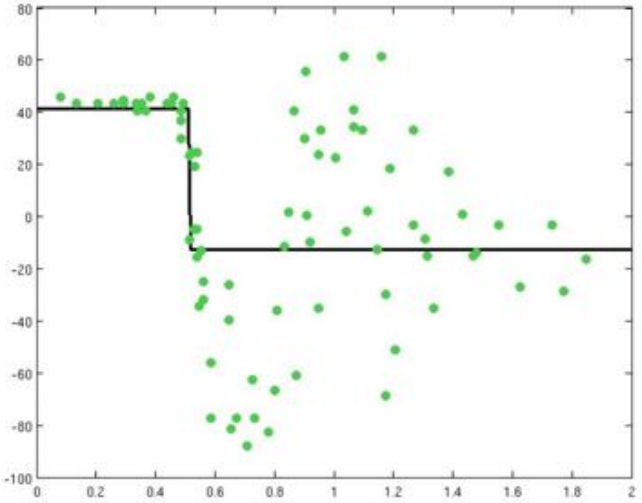
 © 2022 General Assembly

A quick little visual aid of GBoost

Learn a simple predictor…



Then try to correct its errors



 © 2022 General Assembly

Gradient Tree Boosting algorithm, behold!

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ compute

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$

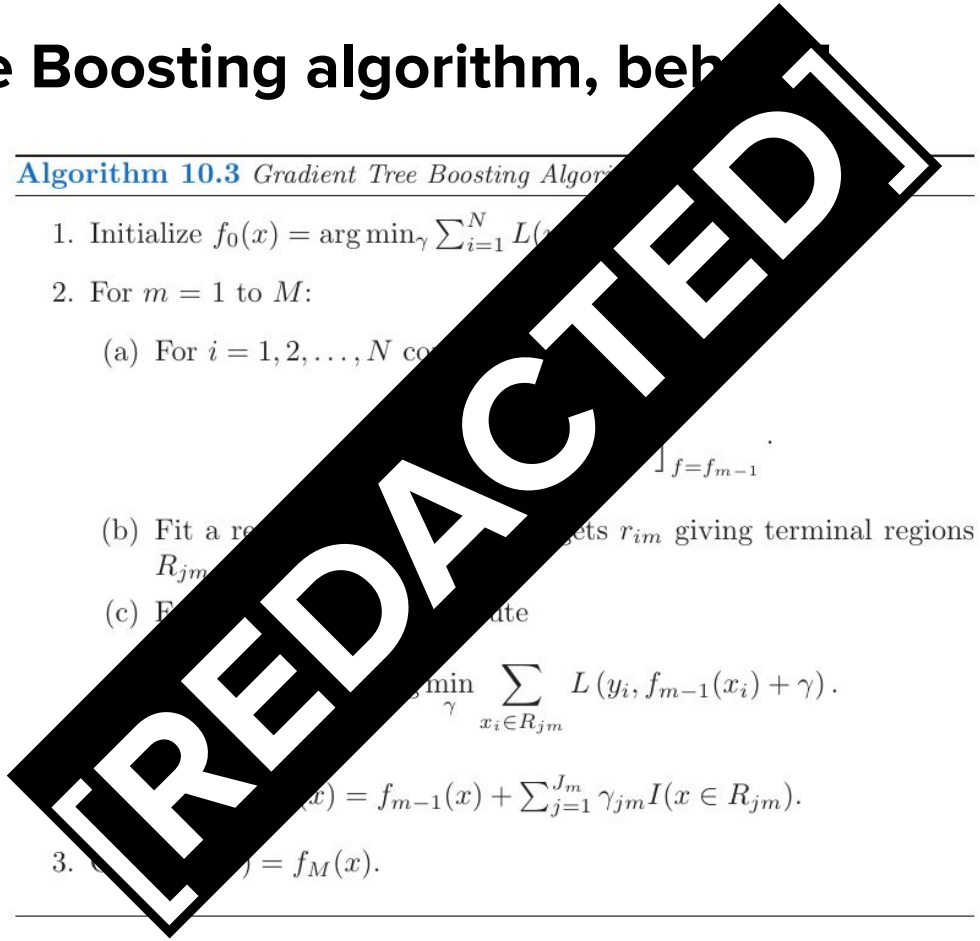
   (b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.
   (c) For $j = 1, 2, \ldots, J_m$ compute

$$\gamma_{jm} = \arg\min_\gamma \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$

   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.
3. Output $\hat{f}(x) = f_M(x)$.

Lifted from ESL

Gradient Tree Boosting algorithm, beh



Algorithm 10.3 *Gradient Tree Boosting Algori*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L($
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ co
   $\Big]_{f=f_{m-1}}$.
   (b) Fit a re ets r_{im} giving terminal regions R_{jm}
   (c) E ute
   $\min_\gamma \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$
   $(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm}).$
3. $) = f_M(x).$

[REDACTED]

Lifted from ESL

A note on these algorithms...

We're getting to the point where the algorithms themselves are far beyond the mathematical scope of our class, and are really most only the interest of sad, lonely academics.

For us, the key things to remember are the gists of why these algorithms do what they do, and the basics of how.

Failing that, we can just ham-fist them in Python, so...

 © 2022 General Assembly

Let's try them out!



 © 2022 General Assembly